

Amrita Vishwa Vidhyapeetham

Chennai Campus

Data Analysis And Algorithms

Week-6

Greedy Algorithms

Job Sequencing

Dhusyanth R

CH.SC.U4CSE24161

Job sequencing:

- 1.) let there be 14 jobs with the profit of
 (22,19,29,28,30,21,27,25,24,26,14,27,19,11)
 deadline(3,3,8,6,7,5,10,4,6,12,13,2,14,1) do job sequencing:
 Solution:

```

job sequencing.cpp
1  #include <stdio.h>
2
3  #define N 14 // Number of jobs
4
5  // Structure for Job
6  struct Job {
7      int id;
8      int profit;
9      int deadline;
10 };
11
12 // Function to sort jobs in descending order of profit
13 void sortJobs(struct Job jobs[]) {
14     int i, j;
15     struct Job temp;
16
17     for(i = 0; i < N - 1; i++) {
18         for(j = 0; j < N - i - 1; j++) {
19             if(jobs[j].profit < jobs[j + 1].profit) {
20                 temp = jobs[j];
21                 jobs[j] = jobs[j + 1];
22                 jobs[j + 1] = temp;
23             }
24         }
25     }
26 }

```

```

28 int main() {
29
30     struct Job jobs[N] = {
31         {1, 22, 3},
32         {2, 19, 3},
33         {3, 29, 8},
34         {4, 28, 6},
35         {5, 30, 7},
36         {6, 21, 5},
37         {7, 27, 10},
38         {8, 25, 4},
39         {9, 24, 6},
40         {10, 26, 12},
41         {11, 14, 13},
42         {12, 27, 2},
43         {13, 19, 14},
44         {14, 11, 1}
45     };
46
47     int i, j;
48
49     // Sort jobs by profit
50     sortJobs(jobs);
51
52     // Find maximum deadline
53     int maxDeadline = 0;
54     for(i = 0; i < N; i++) {
55         if(jobs[i].deadline > maxDeadline)
56             maxDeadline = jobs[i].deadline;
57     }
58
59     int slot[maxDeadline + 1]; // Time slots
60     int totalProfit = 0;
61
62     // Initialize all slots as empty
63     for(i = 0; i <= maxDeadline; i++)
64         slot[i] = -1;
65
66     // Assign jobs to slots
67     for(i = 0; i < N; i++) {
68         for(j = jobs[i].deadline; j > 0; j--) {
69             if(slot[j] == -1) {
70                 slot[j] = jobs[i].id;
71                 totalProfit += jobs[i].profit;
72                 break;
73             }
74         }
75     }
76
77     printf("\nSelected Job Sequence:\n");
78     for(i = 1; i <= maxDeadline; i++) {
79         if(slot[i] != -1)
80             printf("Slot %d -> Job %d\n", i, slot[i]);
81     }
82
83     printf("\nMaximum Profit = %d\n", totalProfit);
84
85     return 0;
86 }
87

```

Output:

```
Selected Job Sequence:
Slot 1 -> Job 6
Slot 2 -> Job 12
Slot 3 -> Job 1
Slot 4 -> Job 8
Slot 5 -> Job 9
Slot 6 -> Job 4
Slot 7 -> Job 5
Slot 8 -> Job 3
Slot 10 -> Job 7
Slot 12 -> Job 10
Slot 13 -> Job 11
Slot 14 -> Job 13

Maximum Profit = 292

-----
Process exited after 0.01382 seconds with return value 0
Press any key to continue . . .
```

Time Complexity:

Time Complexity of Job Sequencing with Deadlines:

- If jobs are sorted first → sorting takes $O(n \log n)$
- Assigning jobs to slots takes $O(n^2)$ in worst case

Overall Time Complexity = $O(n^2)$

(Where n is the number of jobs.)

If advanced methods like Disjoint Set (Union-Find) are used, it can be reduced to:

$O(n \log n)$.

Huffman Coding:

Do DATA ANALYTICS AND INTELLIGENCE LABORATORY in Huffman Coding

Solution:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAX 256
6
7  // Huffman Tree Node
8  struct Node {
9      char data;
10     int freq;
11     struct Node *left, *right;
12 };
13
14 // Create new node
15 struct Node* createNode(char data, int freq) {
16     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
17     newNode->data = data;
18     newNode->freq = freq;
19     newNode->left = newNode->right = NULL;
20     return newNode;
21 }
22
23 // Sort nodes by frequency (ascending)
24 void sort(struct Node* arr[], int n) {
25     for(int i = 0; i < n-1; i++) {
26         for(int j = 0; j < n-i-1; j++) {
27             if(arr[j]->freq > arr[j+1]->freq) {
28                 struct Node* temp = arr[j];
29                 arr[j] = arr[j+1];
30                 arr[j+1] = temp;
31             }
32         }
33     }
34 }
35
36 // Print Huffman Codes
37 void printCodes(struct Node* root, int code[], int top) {
38     if(root->left) {
39         code[top] = 0;
40         printCodes(root->left, code, top+1);
41     }

```

```

43     if(root->right) {
44         code[top] = 1;
45         printCodes(root->right, code, top+1);
46     }
47
48     // Leaf node
49     if(!root->left && !root->right) {
50         printf("%c : ", root->data);
51         for(int i = 0; i < top; i++)
52             printf("%d", code[i]);
53         printf("\n");
54     }
55 }
56
57 int main() {
58
59     char sentence[] = "DATA ANALYTICS AND INTELLIGENCE LABORATORY";
60
61     int freq[MAX] = {0};
62
63     // Count frequency (ignore spaces)
64     for(int i = 0; sentence[i] != '\0'; i++) {
65         if(sentence[i] != ' ')
66             freq[(int)sentence[i]]++;
67     }
68
69     struct Node* arr[MAX];
70     int count = 0;
71
72     // Create nodes for characters present
73     for(int i = 0; i < MAX; i++) {
74         if(freq[i] > 0) {
75             arr[count++] = createNode((char)i, freq[i]);
76         }
77     }
78
79     int n = count:

```

```

81 // Build Huffman Tree
82 while(n > 1) {
83     sort(arr, n);
84
85     struct Node* left = arr[0];
86     struct Node* right = arr[1];
87
88     struct Node* newNode = createNode('$', left->freq + right->freq);
89     newNode->left = left;
90     newNode->right = right;
91
92     arr[0] = newNode;
93     arr[1] = arr[n-1];
94     n--;
95 }
96
97 int code[100], top = 0;
98
99 printf("Huffman Codes for:\n%s\n\n", sentence);
100 printCodes(arr[0], code, top);
101
102 return 0;
103 }

```

Output:

```

Huffman Codes for:
DATA ANALYTICS AND INTELLIGENCE LABORATORY

O : 0000
R : 0001
C : 0010
D : 0011
L : 010
N : 011
T : 100
Y : 1010
S : 10110
B : 101110
G : 101111
E : 1100
I : 1101
A : 111

-----
Process exited after 0.1429 seconds with return value 0
Press any key to continue . . .

```

Time Complexitty:

Time Complexity of Huffman Coding:

- If implemented using a **Min Heap (Priority Queue)** → **$O(n \log n)$**
- If implemented using **simple sorting (like bubble sort)** → **$O(n^3)$**

(Where n is the number of distinct characters.)

Time Complexity = $O(n \log n)$.